

TravExplorer: Cross-Floor Embodied Exploration via Traversability-Aware 3-D Planning

Han Zheng¹, Zhe Chen¹, Yudong Huang¹, Haoran Liu¹, Jinghao Wang¹, Ming Yang¹, and Tong Qin^{1*}

Abstract—Zero-shot Object Navigation (ZSON) has shown promise for open-vocabulary target search in unseen environments, yet most existing systems remain tied to planar representations and single-floor assumptions. These assumptions become inadequate in real buildings, where navigation involves floors, stairs, landings, and vertically overlapping spaces. This article presents TravExplorer, a cross-floor embodied exploration framework that couples zero-shot semantic guidance with traversability-aware 3-D planning. TravExplorer maintains a unified volumetric map that distinguishes occupied structures from robot-reachable support surfaces and extracts traversable frontiers from connected support surfaces, including floors, stairs, and landings. A FOV-aware active perception strategy further resolves incomplete observations during cross-floor traversal. To reduce semantic-reasoning latency, a lightweight guidance module aligns a probabilistic instance map from online open-vocabulary segmentation with a spatial value map from fast image-to-text matching. Based on these geometric and semantic memories, a hierarchical planner performs target-aware frontier touring over object hypotheses, traversable frontiers, and stair landmarks, and generates executable cross-floor motions through foothold-guided 3-D search and vertically constrained local trajectory optimization. Experiments over 4,195 simulated episodes on HM3D and MP3D demonstrate consistent advantages over representative ObjectNav baselines. Fifty real-world trials on a Unitree Go2 further validates open-vocabulary target search across single-floor and cross-floor indoor environments without prior maps or human intervention. The code will be released at <https://github.com/wuyi2121/TravExplorer>.

Index Terms—Zero-shot object navigation, embodied exploration, quadruped robots, traversability mapping.

I. INTRODUCTION

Object navigation (ObjectNav) requires a robot to search for a target object in an unseen environment and stop near a valid instance. In real service and inspection scenarios, a simple query such as “find the fire extinguisher” may require the robot to explore rooms, corridors, stairs, and landings while reasoning from partial observations. Recent large language models (LLMs) and vision-language models (VLMs) have made zero-shot ObjectNav (ZSON) increasingly practical by enabling open-vocabulary perception and commonsense-guided exploration [1]–[4]. However, most existing systems are still developed under planar and single-floor assumptions. This mismatch becomes critical in real buildings, where the robot and the target may lie on different floors connected by stairs or other non-planar support surfaces. Fig. 1 illustrates the cross-floor search scenario considered in this work.

¹Shanghai Jiao Tong University, Shanghai, China. {hanzheng, qin-tong}@sjtu.edu.cn.

* is the Corresponding author. This work was supported by the Natural Science Foundation of Shanghai (Grant No. 24ZR1435600).

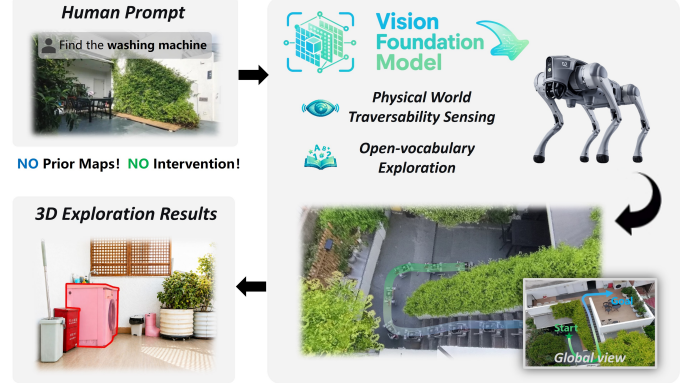


Fig. 1: Motivating scenario of TravExplorer. Given an open-vocabulary object query, the quadruped robot explores an unknown building without prior maps or human intervention. TravExplorer couples foundation-model perception with 3-D traversability reasoning, enabling the robot to search across floors, stairs, and landings while reaching executable viewpoints for target localization.

A common ZSON pipeline constructs a 2-D occupancy or semantic map, extracts frontiers on the boundary of known free space, and ranks candidate frontiers using VLM or LLM cues [3, 5, 6]. This paradigm has shown strong performance in planar environments, but two issues remain for long-horizon cross-floor search. First, local semantic scores from early or occluded views can be unstable, leading to redundant backtracking or false stopping [5, 6]. Second, repeatedly querying large models for frontier or viewpoint selection introduces non-negligible latency and computation cost on robotic platforms [7, 8]. Cross-floor search amplifies both issues because semantic evidence must be accumulated and reused across larger, vertically connected spaces.

The geometric difficulty is more fundamental. For a ground or legged robot, floors, stairs, ramps, and landings are traversable support surfaces, although they appear as occupied geometry in a volumetric map. A pure occupancy representation may therefore remove the very surfaces on which the robot should move. In contrast, a top-down projection preserves planning efficiency but collapses vertically separated floors and breaks stair connectivity. Recent floor-aware ZSON methods introduce multi-floor reasoning [9, 10], yet their layer-wise maps and discrete floor transitions still fragment the underlying 3-D traversable space. Existing 3-D navigation frameworks can preserve multi-layer geometry [11], but they usually assume a known or sufficiently reconstructed map and do not address online semantic exploration.

These observations suggest that cross-floor ObjectNav is neither a semantic target-search problem nor a 3-D path-

planning problem. The robot must discover traversable support surfaces online, maintain revisitable connectivity across floors, use semantic cues without excessive large-model calls, and convert exploration decisions into executable motion under limited field of view. In particular, stair regions require active perception, since incomplete observations may disconnect the very transition that enables cross-floor search.

To this end, we propose *TravExplorer*, a cross-floor embodied exploration system for quadruped robots in 3-D traversable spaces. *TravExplorer* maintains a volumetric occupancy map with a semantic traversability layer, representing floors, stairs, and landings in a unified metric frame. It extracts frontiers directly on connected traversable support surfaces and uses active perception to complete uncertain stair observations. For semantic guidance, *TravExplorer* fuses online open-vocabulary segmentation, probabilistic instance memory, and lightweight image-text matching. For motion generation, a hierarchical planner couples target-aware frontier touring, foothold-guided 3-D graph search, and local trajectory optimization.

The main contributions are summarized as follows:

- (i) We pioneer a traversable frontier-based exploration paradigm for ground robots, which unifies 2D ray casting for efficiency and 3D semantic traversability for multi-floor awareness, while leveraging active perception to mitigate FOV-limited occlusions.
- (ii) We design spatial semantic guidance that couples a probabilistic instance map from open-vocabulary semantic segmentation with a spatial value map from fast image-to-text matching, enabling zero-shot search to focus on semantically promising regions for efficient exploration.
- (iii) We develop a hierarchical cross-floor exploration planner that integrates TSP-based global tours, foothold-guided 3D path search, an execute-review mechanism, and vertically constrained local planning, enhancing both search efficiency and traversal robustness.
- (iv) We conduct extensive evaluations, including 50 real-world experiments and 4,195 simulated episodes on HM3D and MP3D benchmarks, achieving state-of-the-art performance. To the best of our knowledge, this is the first system that efficiently handles both single-floor and cross-floor object navigation through 3D traversability mapping.

II. RELATED WORK

A. Autonomous Exploration.

Autonomous exploration is commonly formulated as selecting informative viewpoints that expand the known map while keeping the motion executable. Frontier-based exploration [12] identifies boundaries between known and unknown regions as exploration targets, providing a simple and scalable basis for mobile robots. Efficient dense frontier detection [13] accelerates frontier extraction on 2-D graph-SLAM submaps, FAEL [14] improves large-scale mobile-robot exploration efficiency, and region partitioning [15] reduces redundant long-range transfers. These methods are effective for planar occupancy or projected maps, but the projection collapses stairs and vertically overlapping corridors into ambiguous 2-D cells.

For richer spatial coverage, another line of work performs exploration in 3-D volumetric spaces. FUEL [16] maintains

an incremental frontier information structure and uses hierarchical planning to generate fast UAV motions. FALCON [17] further introduces connectivity-aware space decomposition and coverage-path guidance, aligning local frontier visitation with a global intention over unexplored space. 3-D Active Metric-Semantic SLAM [18] incorporates localization uncertainty into active metric-semantic reconstruction, EPIC [19] uses lightweight LiDAR point-cloud observation maps for large-scale UAV exploration, and FLARE [20] partitions unknown regions to guide fast exploration. Although these explorers better preserve 3-D topology and reduce revisitation, their candidate viewpoints are generated in free space and are not required to lie on ground-reachable support surfaces.

Ground-robot exploration must additionally reason about terrain traversability, stability, and collision clearance. SMUG Planner [21] performs safe multi-goal planning with robot-specific traversability constraints, LRAE [22] extracts large safe exploration regions on uneven terrain, and Portable Planner [23] improves exploration robustness in unstructured environments. AAGE [24] uses aerial bird’s-eye-view priors to assist ground robots in large unknown scenes. However, these systems are mainly designed for outdoor rough terrain or 2.5-D elevation-map settings. They do not explicitly address indoor multi-floor exploration, where different floors may overlap in the horizontal plane and vertical transitions must remain continuously connected to the traversable map.

B. Cross-Floor Trajectory Planning.

Ground navigation beyond planar workspaces is often built on terrain representations that expose height and traversability. Probabilistic terrain mapping [25] estimates grid-based elevation with uncertainty bounds under drifting proprioceptive localization, and GPU elevation mapping [26] accelerates smoothing, inpainting, and segmentation for locomotion and navigation. Perceptive locomotion through nonlinear MPC [27] integrates terrain perception into whole-body control, while uneven-terrain trajectory planning [28] uses terrain pose mapping to couple traversability assessment with car-like robot dynamics. Such 2.5-D maps are compact and effective for slopes, steps, and rough ground. However, they normally store one dominant surface per grid cell, which makes them unreliable in indoor buildings with overhangs, stair voids, and vertically overlapping floors.

To extend ground navigation to multi-layer structures, recent methods preprocess 3-D geometry into lower-dimensional but structured planning spaces. Beyond-2-D trajectory generation [29] constructs an R^3 penalty field so that active and passive height variations can be optimized in a continuous trajectory space. PCT [11] analyzes point clouds tomographically, generating slices that encode ground and ceiling elevations while evaluating traversability according to robot motion capabilities. Traversable Planes [30] exploits the fact that architectural spaces are often composed of ground, slopes, and stairs, and builds a lightweight plane graph through segmentation, merging, classification, and connection. These representations significantly reduce the complexity of 3-D search, yet they are mainly designed for goal-directed planning after sufficient geometric observations have been acquired.

Recent robot-specific frameworks further move from offline or goal-directed planning toward online navigation and exploration in complex 3-D terrain. Real-time multilevel terrain-aware planning [31] introduces an implicit global map and local configuration-stability evaluation for large-scale rough terrains, while TiFA [32] provides a terrain-informed navigation framework for articulated tracked robots in rescue missions. These methods show that online terrain reasoning is feasible beyond prebuilt maps. However, they often rely on specialized robot morphologies, task-specific terrain representations, or carefully designed traversability segmentation, which limits their generality for cross-floor ObjectNav.

C. Zero-Shot Object Navigation.

ObjectNav requires an agent to find a target object category in an unseen environment, coupling semantic recognition with long-horizon exploration. Early modular methods, such as SemExp [33] and PONI [34], rely on semantic maps or learned object-location priors to guide exploration, while ZSON [35] extends ObjectNav to unseen categories by aligning language goals with visual observations. More recently, foundation-model-based approaches have shifted zero-shot ObjectNav toward open-vocabulary perception and explicit semantic reasoning. Among them, VLFM [3] is particularly relevant to our work, as it constructs language-grounded value maps over frontier candidates to guide semantic exploration. SG-Nav [4] prompts LLMs with online 3-D scene graphs, FOM-Nav [36] organizes frontier-object maps for object-goal search, and OpenFrontier [37] grounds frontiers with visual-language cues. ApexNav [5] adaptively switches between semantic reasoning and geometry-based exploration according to the distribution of semantic cues, while using target-centric semantic fusion to reduce false detections. STRIVE [6] builds a multi-layer representation of viewpoints, objects, and rooms, SysNav [7] decouples reasoning, planning, and control, and USS-Nav [8] constructs a unified spatio-semantic scene graph for lightweight UAV navigation. These methods improve semantic guidance and reduce unnecessary model queries, but their navigation backbones are still largely based on 2-D frontiers, room-level abstractions, or embodiment-specific motion layers.

Multi-floor zero-shot ObjectNav is a more recent direction. MFNP [9] pioneers multi-floor ZSON by introducing a multi-floor navigation policy, multimodal LLM reasoning, and inter-floor navigation through stairs. However, its stair traversal mechanism resets the map after floor transition, discards previous exploration memory, and marks the stair entrance to enforce one-way movement. ASCENT [10] improves online floor-aware navigation with a multi-floor abstraction, stair-aware obstacle mapping, cross-floor topology modeling, and LLM-driven coarse-to-fine exploration. It maintains separate floor-wise maps and switches among them, alleviating memory loss but still breaking the continuity of the 3-D environment into discrete 2-D layers. Moreover, floor transitions remain dependent on predefined rules, limiting adaptation to diverse stair geometries. TravExplorer instead maintains a unified 3-D traversability map and couples zero-shot semantic guidance with continuous cross-floor motion planning.

III. SYSTEM OVERVIEW

A. Problem Formulation

We first define the task interface, observations, actions, and success criterion used throughout the paper. The ObjectNav task [38] involves the agent navigating in a previously unseen environment to find an instance of the target object category (e.g., ‘chair’). The target-object query set is represented by $Q_{\text{obj}} = \{q_1, \dots, q_n\}$, and the scene set is represented by $S = \{s_1, \dots, s_m\}$. Each episode begins with the agent being initialized at state $\mathbf{x}_i$ in the scene s_i and receiving the target description q_i . Thus, an episode can be denoted as $E_i = (s_i, q_i, \mathbf{x}_i)$. At each time step t , the agent has state $\mathbf{x}^t$, gets the observation Z^t from the environment, and takes an action. At time t , the observation is denoted as $Z^t = \{I^t, D^t, \mathbf{T}_t\}$, where I^t and D^t are the egocentric RGB and depth images captured by a limited-FOV RGB-D camera (79°), and $\mathbf{T}_t$ is the odometry-derived pose relative to the starting pose. The action space consists of six discrete actions, including `move_forward`, `turn_left`, `turn_right`, `look_up`, `look_down`, and `stop`. The `move_forward` action moves the agent 25 cm, while the `turn_left`, `turn_right`, `look_up`, and `look_down` actions rotate the agent by 30° . The `stop` action is used when the agent believes it has found the target object. An episode is considered successful if the agent takes the `stop` action when it is close enough to the target. Each episode has a maximum limit of 500 time steps.

B. The TravExplorer Architecture

As shown in Fig. 2, TravExplorer is organized as a closed-loop architecture that couples traversability-aware mapping, lightweight semantic guidance, and hierarchical exploration planning. At each step, posed RGB-D observations are fused into a volumetric map whose traversability layer is updated from both geometric ray casting and semantic traversability cues. The resulting map supports incremental obstacle inflation and frontier extraction directly on connected support surfaces, allowing floors, stairs, and landings to be represented within a unified 3-D planning space. In parallel, the semantic guidance module accumulates open-vocabulary detections into a probabilistic instance map and projects image-text relevance onto reachable regions to provide target and stair-related evidence. The planning module then reasons over target instances, traversable frontiers, and stair landmarks, selects an admissible exploration goal, searches for a 3-D traversable path, and executes the path with online review and replanning. This design keeps high-level semantic decisions grounded in physically executable traversability, which is essential for cross-floor object navigation with a quadruped robot.

IV. 3-D TRAVERSABLE FRONTIERS

This section introduces the 3-D traversable-frontier representation shown in Fig. 3, which converts raw RGB-D observations into executable exploration candidates in 3-D traversable space. The representation consists of adaptive traversability mapping, incremental obstacle inflation, online frontier updating, and FOV-aware active perception.

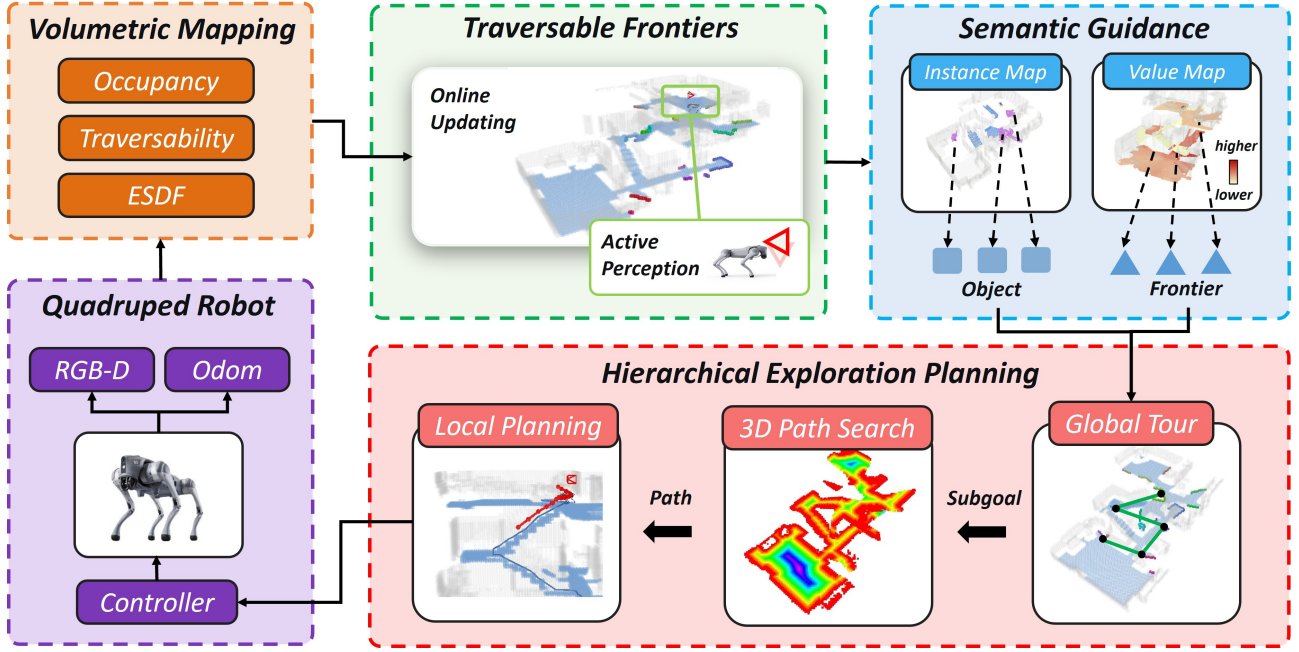


Fig. 2: System overview of TravExplorer. The quadruped robot receives RGB-D and odometry observations and incrementally maintains volumetric occupancy, traversability, and ESDF layers. Traversable frontiers are extracted on connected reachable support surfaces and updated online with active perception. In parallel, the dual-semantic guidance module builds a target-centric instance map and a spatial value map to associate open-vocabulary object evidence with reachable frontier regions. The hierarchical exploration planner then performs global frontier touring, traversability-aware 3-D path search, and local planning, closing the loop by sending executable commands to the robot controller.

A. Adaptive Traversability Mapping

TravExplorer maintains a mode-adaptive traversability layer that is efficient for regular planar exploration while remaining complete enough for non-planar floor transitions. Given an RGB-D observation at time t , denoted by $Z^t = \{I^t, D^t, \mathbf{T}_t\}$, where $\mathbf{T}_t$ is the camera pose in the world frame, the system maintains a voxel map M^t that jointly encodes occupancy and traversability. Each voxel $v_i \in M^t$ has a center position $\mathbf{p}_i$ and an occupancy log-odds value l_i^t . The map maintains two traversability sources: a geometrically traversable set M_g^t and a semantically traversable set M_s^t . The former is updated by efficient 2-D ray casting on the support plane, while the latter is obtained from open-vocabulary segmentation and is mainly used for non-planar structures such as stairs.

The depth image D^t is first back-projected into a point cloud P^t and fused into the occupancy using standard log-odds updates. Voxels with $l_i^t \geq l_{\text{occ}}$ are treated as occupied. The occupied voxels are further inflated according to the robot body size, forming the inflated obstacle set O_{inf}^t to keep a safety margin during traversability update and planning.

Geometric traversability is estimated on the robot support plane. Let z_r^t denote the height of the current support plane. Points within the robot collision-height range, i.e., $h_{\min} < p_z - z_r^t < h_{\max}$, are projected onto the plane $z = z_r^t$. For each projected obstacle point, a 2-D ray is cast from the point to the robot support state $\mathbf{x}_r^t$. Voxels traversed by each ray are marked as geometrically traversable and inserted into M_g^t , while occupied voxels are skipped. This implements 2-D ray casting within the 3-D voxel map.

Semantic traversability is obtained from open-vocabulary segmentation. Given robot-traversable categories (e.g. floor,

stair, and rug), the RGB image I^t is segmented to extract traversable regions. Corresponding pixels are back-projected using D^t , transformed into the world frame by $\mathbf{T}_t$, and the associated voxels are inserted into M_s^t . Open-vocabulary semantic traversability further recovers stairs and ramps that are treated as obstacles by 2-D ray casting, and supports unstructured open-set terrains such as grass.

To balance mapping efficiency and cross-floor completeness, the system switches between geometric and semantic traversability modes. Let $m^t \in \{g, s\}$ denote the mapping mode at time t , where g and s indicate geometric and semantic modes, respectively. During regular exploration, $m^t = g$, and the planner uses the geometric traversability induced by 2-D ray casting. When stairs are stably detected over N_{enter} consecutive frames, or when the high-level planner requests navigation to a stair entrance landmark, the system switches to $m^t = s$, allowing semantic segmentation to provide traversability over non-planar support regions. The final traversable map used by the planner is obtained by selecting the traversability source consistent with the current mode and removing inflated obstacles:

$$M_{\text{trav}}^t = \left(\bigcup_{\alpha \in \{g, s\}} (M_{\alpha}^t \mid m^t = \alpha) \right) \setminus O_{\text{inf}}^t. \quad (1)$$

The system switches back to geometric mode only after the robot remains sufficiently far from the stair region for N_{exit} consecutive frames. This hysteresis avoids frequent mode oscillations caused by intermittent stair observations.

B. Incremental Obstacle Inflation

Given the traversability map, the inflation module constructs a planning-safe obstacle layer while avoiding unnecessary

inflation of walkable support surfaces. Uniform 3-D inflation is not suitable for ground robots, since it may incorrectly penalize traversable support regions. We therefore inflate only non-traversable occupied voxels. In addition, we design an anisotropic inflation neighborhood. Specifically, the robot is approximated by the center of its bottom support plane. The inflation kernel is defined around this reference point, with a fixed radius in the xy -plane and a downward-only range along the z -axis. After each occupancy and traversability update, the system tracks only voxels whose states have changed. The voxels include occupied-to-free transitions and non-traversable-to-traversable transitions:

$$C^t = (O^{t-1} \setminus O^t) \cup (M_{\text{trav}}^t \setminus M_{\text{trav}}^{t-1}). \quad (2)$$

For each changed voxel $v_i \in C^t$, previous inflated labels inside its inflation neighborhood $N_{\text{inf}}(v_i)$ are cleared. The operation removes obsolete inflated regions caused by newly freed voxels or newly traversable support regions. Re-inflation is restricted to the current local update region Ω^t and non-traversable voxels. Specifically, the re-inflated region is

$$R_{\text{inf}}^t = \bigcup_{v_i \in S_{\text{inf}}^t} N_{\text{inf}}(v_i), \quad S_{\text{inf}}^t = \Omega^t \cap O^t \setminus M_{\text{trav}}^t. \quad (3)$$

After clearing and re-inflation, the inflated obstacle set O_{inf}^t is updated incrementally, while ESDF and frontier updates are then triggered. Our incremental inflation strategy has a time complexity of $O(n)$, where n is the number of changed voxels induced by both occupancy and traversability transitions, avoiding global inflation over the entire 3-D map.

C. Traversable Frontier Updating

The safe traversability map further enables frontier candidates that are both informative for exploration and reachable by the robot. To support exploration in 3-D traversable spaces, we define frontiers on robot-reachable support surfaces. Unlike conventional frontiers [16], which are known-free voxels adjacent to unknown space, traversable frontiers describe the boundary between reachable traversable surfaces and locally unexplored or temporarily non-traversable regions.

Frontiers are extracted from the traversable map M_{trav}^t . A voxel $v_i \in M_{\text{trav}}^t$ is regarded as a traversable frontier candidate if it satisfies two additional conditions. First, v_i must have sufficient headroom. No traversable voxel should appear within K_h layers above v_i , which avoids extracting frontiers inside vertical structures or overlapping traversable layers. Second, v_i must lie on the boundary of the current traversable surface. Let $N_4(v_i)$ denote the horizontal four-neighborhood of v_i . The voxel v_i is selected if there exists a neighbor $v_j \in N_4(v_i)$ whose vertical column is not traversable. This places the frontier at the boundary between the reachable support surface and unexplored or temporary non-traversable space. The resulting candidates are grouped by 26-neighbor connectivity. A traversable frontier cluster is defined as $F_i = \langle C_i, \mathbf{p}_i, B_i \rangle$, where $C_i = \{v_1, v_2, \dots, v_{n_i}\}$ denotes the set of frontier voxels in cluster i , $\mathbf{p}_i \in \mathbb{R}^3$ is the geometric centroid of C_i , and B_i is the axis-aligned bounding box (AABB) of the cluster. The centroid $\mathbf{p}_i$ is used as the representative point for goal selection and path-cost evaluation. Clusters smaller than a predefined size threshold are discarded.

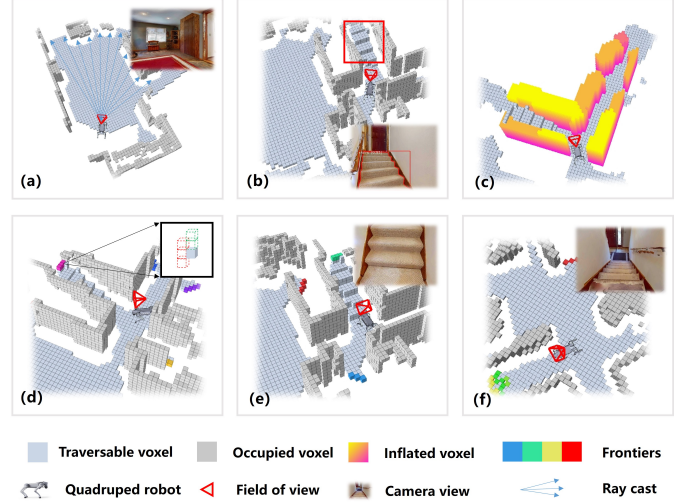


Fig. 3: 3-D traversable frontier representation. Blue, gray, and inflated voxels denote traversable support surfaces, occupied structures, and inflated collision regions, respectively, while colored clusters indicate traversable frontiers. TravExplorer extracts and updates frontiers only on robot-reachable support surfaces, with ray-casting-based updates and FOV-aware active perception to support executable exploration in cluttered and cross-floor environments.

As the traversable map M_{trav}^t is updated by new sensor measurements, the updated region is recorded by its AABB B_m^t . Frontier updating is performed only around this affected region to support real-time updates, following [16]. The algorithm first traverses all existing frontier clusters and returns those whose AABBs B_i intersect with B_m^t . These affected clusters are removed, and their visited flags are reset. New frontier clusters are then extracted in the expanded neighborhood of B_m^t by region growing with 26-neighbor connectivity. Clusters outside the affected region are kept unchanged, avoiding global frontier re-extraction over the entire 3-D map.

D. FOV-Aware Active Perception

Limited forward-looking camera FOV motivates active perception around stair transitions and close-range frontier regions. Nearby traversable regions may remain unobserved during exploration. This issue is common when the robot approaches staircases from narrow corridors, where the first stair step may fall outside the forward-looking view. As a result, nearby frontier clusters remain active even when the robot is already close to the corresponding region.

To handle this issue, we introduce an active perception strategy. For each active frontier cluster $F_i \in F_{\text{act}}$, its distance to the robot is computed as $d_i = \|\mathbf{p}_i - \mathbf{x}_r^t\|$, where $\mathbf{p}_i$ is the representative point of F_i . If $d_i < d_{\text{blind}}$, the cluster is regarded as a blind-spot frontier. The robot then actively pitches the camera downward to observe the nearby unscanned region and update the traversable map. If the hidden traversable voxels are observed, the corresponding frontier is removed by the local frontier update. Otherwise, the frontier is regarded as an irreducible frontier and moved from the active set F_{act} to the dormant set F_{dor} to avoid repeated ineffective actions.

Descending stairs also require active perception, since they may be completely outside the forward-looking view. We introduce a virtual ground plane Π_{vg} below the robot. During

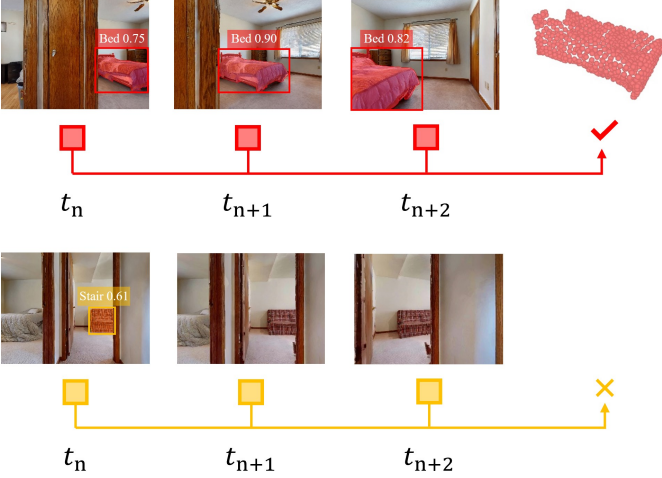


Fig. 4: Target-centric instance memory for robust open-vocabulary perception. TravExplorer accumulates temporally consistent object detections over time and back-projects them into a unified 3-D instance representation. Persistent detections of the queried object, such as the bed shown in the top row, are fused into a valid target instance, while transient or inconsistent semantic responses, such as the false stair detection in the bottom row, are rejected. This temporal instance memory reduces single-view false positives and provides more reliable semantic evidence for target-guided exploration.

exploration, if the number of observed points below Π_{vg} exceeds a threshold N_{down} , the robot triggers a downward-looking behavior. This allows the robot to verify descending staircases and maintain a downward view during stair descent.

V. SPATIAL SEMANTIC GUIDANCE

Semantic guidance provides target awareness on top of traversable-frontier exploration. Instead of invoking large-model reasoning at every planning step, TravExplorer maintains two lightweight map-level memories: a probabilistic instance map for persistent object and landmark hypotheses, and a spatial value map that projects image-text relevance onto reachable support surfaces.

A. Probabilistic Instance Map

Single-frame open-vocabulary detections are accumulated into persistent 3-D semantic hypotheses before being used for planning. Open-vocabulary perception provides zero-shot recognition ability, but single-frame predictions are often unstable due to false positives, missed detections, occlusions, and viewpoint changes. We maintain a probabilistic semantic instance map. The object branch accumulates target-object hypotheses for goal-directed navigation, while the landmark branch accumulates navigation-related semantic landmarks, such as stairs, for cross-floor guidance.

The query set is defined as $Q = Q_{obj} \cup Q_{land}$, where Q_{obj} contains target-object queries and Q_{land} contains traversability-related landmark queries. At time t , for each query $q \in Q$, the open-vocabulary segmentation module extracts an image region R_q^t from I^t . Pixels in R_q^t are back-projected using D^t to obtain a semantic point cloud P_q^t . Each detection is represented as $d_q^t = (P_q^t, s_q^t, q)$, where s_q^t is the detection confidence.

The instance map contains semantic clusters Y^t . Each cluster $y \in Y^t$ stores a semantic label q_y , a fused point cloud P_y^t ,

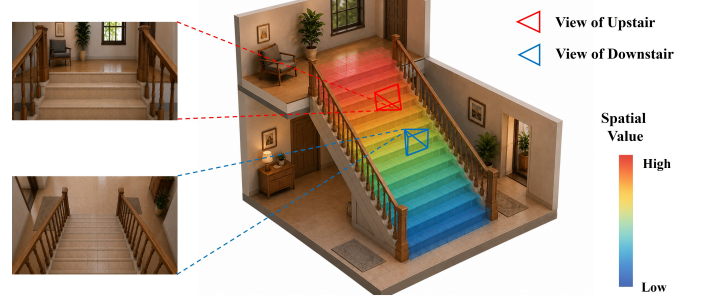


Fig. 5: Traversability-aware spatial value map on a staircase. Image-text relevance from upstairs and downstairs views is projected onto reachable stair surfaces and fused into a 3-D value map. Higher values indicate regions with stronger target-related evidence, enabling direction-aware semantic guidance for cross-floor exploration.

a fused confidence γ_y^t , and an observed point count n_y^t . A detection is associated only with clusters of the same label, and the association is accepted when their 3-D centroid distance is below δ_{asso} . For an associated cluster, the current detection is treated as positive evidence. Following [5], the point cloud is integrated with voxel downsampling $D(\cdot)$, and the confidence is updated by weighted fusion:

$$P_y^t = D(P_y^{t-1} \cup P_q^t), \quad n_y^t = n_y^{t-1} + n_q^t, \quad (4)$$

$$\gamma_y^t = \frac{n_y^{t-1} \gamma_y^{t-1} + n_q^t s_q^t}{n_y^t}, \quad n_q^t = |P_q^t|. \quad (5)$$

If no valid association is found, a new cluster is initialized from the current detection. To suppress stale false positives, visible but unmatched clusters are updated with negative evidence. If a cluster y lies inside the current field of view but receives no associated detection, its visible support $\tilde{n}_y^t$ is added as zero-confidence evidence:

$$n_y^t = n_y^{t-1} + \tilde{n}_y^t, \quad \gamma_y^t = \frac{n_y^{t-1} \gamma_y^{t-1}}{n_y^t}. \quad (6)$$

This decreases the confidence of hypotheses that are repeatedly observed but not consistently detected, while preserving instances supported by stable multi-frame observations.

The two branches provide different candidates to the hierarchical planner. Clusters from Q_{obj} are inserted into O_{tar} , while clusters from Q_{land} are inserted into O_{stair} . For each query q , reliable candidates are selected according to the accumulated belief $\gamma_y^t n_y^t$. A cluster is used by the planner only when its fused confidence exceeds a predefined threshold; otherwise, it is retained as a tentative hypothesis for later verification.

B. Spatial Value Map

The spatial value map assigns language-conditioned utility to reachable traversable voxels so that frontier selection can exploit semantic evidence before a reliable target instance is confirmed. Frequent VLM or LLM reasoning in the control loop introduces substantial latency. Following VLFM [3], we use BLIP-2 [39] as a fast image-text matching model for semantic guidance. To guide target-driven exploration in 3-D traversable space, we maintain a spatial value map $V^t = (s^t, r^t)$ over robot-reachable

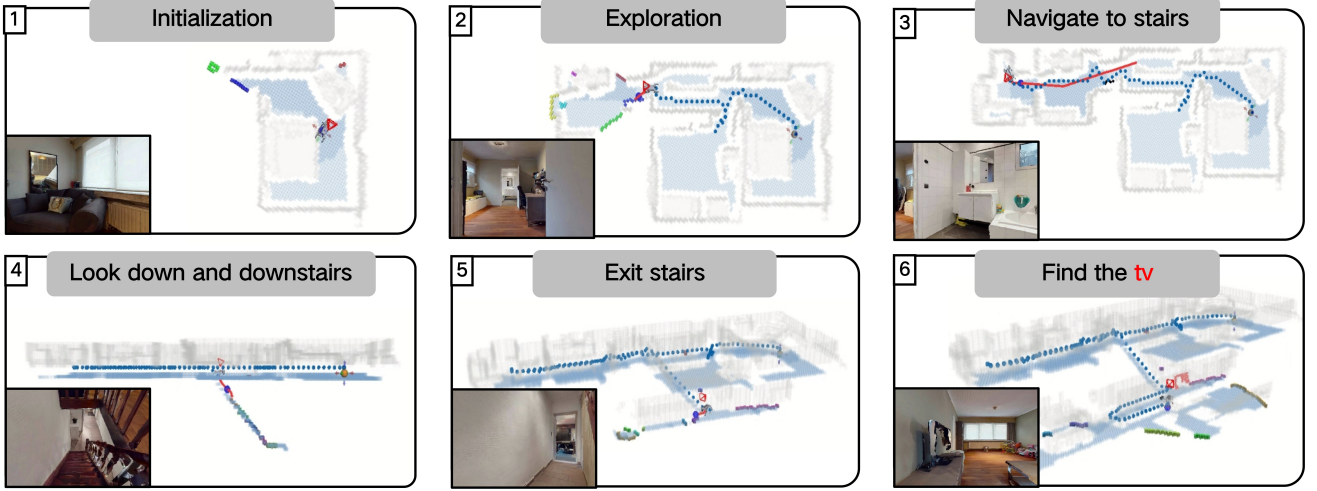


Fig. 6: Cross-floor exploration process. (a) The robot initializes the map and frontier candidates from early RGB-D observations. (b) TravExplorer expands the known region by following reachable traversable frontiers. (c) The planner detects a stair-related semantic landmark and navigates toward the stair region. (d) The robot actively looks downward to complete FOV-limited stair observations and update traversability. (e) After crossing the stairs, the system preserves the unified 3-D map and continues exploration on the new floor. (f) Guided by open-vocabulary semantic evidence, the robot reaches the queried target, tv .

support regions, where $s_k^t \in [0, 1]$ is the accumulated spatial semantic value of voxel v_k , and $r_k^t \in [0, 1]$ is its accumulated reliability. This map integrates semantic evidence in 3-D and mitigates the limited spatial scene understanding of VLMs.

At time t , BLIP-2 produces an image-text relevance score $\hat{s}^t$ for the current RGB observation. This score is projected only onto visible traversable voxels. For each visible voxel v_k , the observation weight $\omega_k^t = \cos^2(\pi\theta_k^t/\theta_{FOV})$ is computed from its horizontal angular offset θ_k^t to the camera heading, assigning larger weights to voxels near the image center and smaller weights near the FOV boundary. The semantic value and reliability are updated by confidence-weighted fusion:

$$s_k^t = \frac{r_k^{t-1}s_k^{t-1} + \omega_k^t\hat{s}^t}{r_k^{t-1} + \omega_k^t}, \quad r_k^t = \frac{(r_k^{t-1})^2 + (\omega_k^t)^2}{r_k^{t-1} + \omega_k^t}. \quad (7)$$

This update accumulates language-conditioned evidence from multiple viewpoints and suppresses unstable single-frame predictions. As illustrated in Fig. 5, semantic evidence from upstairs and downstairs viewpoints is fused on the corresponding reachable stair surfaces, forming a coherent spatial belief for downstream planning. For a candidate node, its semantic utility is computed from the maximum value in its local traversable neighborhood. A higher value indicates that the candidate frontier is more likely to guide the robot toward the target object.

VI. HIERARCHICAL EXPLORATION PLANNING

Given the traversable map and semantic memories, the planner must decide both which exploration candidate to visit and how to reach it safely. TravExplorer addresses this with a hierarchical pipeline: an open-tour selects among target instances, traversable frontiers, and stair landmarks; a 3-D graph search provides a route over traversable voxels; an execute-review loop monitors progress; and a local trajectory optimizer generates safe commands for real-world deployment.

A. Chain of Exploration

Goal selection follows a hierarchical open-ATSP formulation. At each replanning step, the robot first maintains a unified candidate pool that includes the current robot state, semantic instances, and frontier clusters:

$$X = \{\mathbf{x}_r\} \cup O \cup F. \quad (8)$$

Here, $\mathbf{x}_r$ is the current robot state. The semantic candidate set is $O = O_{\text{tar}} \cup O_{\text{stair}}$, where O_{tar} contains reliable target-object instances and O_{stair} contains stair-entry landmarks. The frontier candidate set is $F = F_g \cup F_s$, where F_g and F_s denote geometric and semantic frontiers on the current floor, respectively.

Instead of optimizing over all candidates simultaneously, the planner activates one admissible subset according to a lexicographic exploration priority:

$$A = \begin{cases} O_{\text{tar}}, & O_{\text{tar}} \neq \emptyset, \\ F, & O_{\text{tar}} = \emptyset, F \neq \emptyset, \\ O_{\text{stair}}, & O_{\text{tar}} = \emptyset, F = \emptyset. \end{cases} \quad (9)$$

The actual ATSP node set is then

$$X_A = \{\mathbf{x}_r\} \cup A. \quad (10)$$

This formulation decouples candidate maintenance from goal optimization: navigation cues are maintained in a unified pool, whereas the open ATSP is solved only over the subset activated by the current exploration stage.

Navigation is formulated as an open Asymmetric Traveling Salesperson Problem (ATSP) [40] over the active node set X_A . Let A denote the active candidate nodes. The cost matrix is constructed as

$$C_G = \begin{bmatrix} c_{r,r} & c_{r,A}^\top \\ c_{A,r} & C_{A,A} \end{bmatrix}, \quad (11)$$

where $c_{r,r} = 0$ and $c_{A,r} = \mathbf{0}$. This removes the cost of returning to the current robot state and yields an open tour.

For two active candidate nodes v_i and v_j , the candidate-to-candidate cost is

$$C_{A,A}(i, j) = \begin{cases} 0, & i = j, \\ J_{\text{geo}}(v_i, v_j) + J_{\text{front}}(v_j), & i \neq j. \end{cases} \quad (12)$$

The cost from the current robot state to an active candidate is

$$c_{r,A}(j) = J_{\text{geo}}(\mathbf{x}_r, v_j) + J_{\text{front}}(v_j). \quad (13)$$

Here, J_{geo} is the shortest traversable path cost on the current traversability map. If no feasible path exists, J_{geo} is set to M_∞ . The frontier preference term is defined as

$$J_{\text{front}}(v) = \begin{cases} \lambda_g, & v \in F_g, \\ \lambda_s, & v \in F_s, \\ 0, & v \notin F, \end{cases} \quad 0 \leq \lambda_g < \lambda_s. \quad (14)$$

This term provides a soft intra-frontier preference: when the active set is the current-floor frontier set F , geometric frontiers are preferred over semantic frontiers, while semantic frontiers serve as complementary candidates for non-planar traversable regions that may otherwise cause redundant revisits.

The hierarchical activation implements the exploration priority before ATSP optimization. Target objects dominate all other navigation cues. If no reliable target is available, the planner selects from the current-floor frontier set F . Within F , J_{front} softly prefers geometric frontiers over semantic frontiers. Stair landmarks are activated only after current-floor frontiers are exhausted. They trigger a cross-floor transition stage, during which semantic frontiers guide the robot through non-planar stair regions where geometric frontiers are not continuously defined. Once the robot reaches a new floor, the frontier candidate pool is updated and the same decision process is repeated, yielding the exploration chain.

B. 3-D Path Search

After a navigation goal is selected, the planner computes a physically executable route on the traversable support graph rather than in the full 3-D free-space grid. The search is performed on the traversable map M_{trav}^t , where inflated obstacles have been removed. Compared with 26-neighbor expansion in dense 3-D voxel space, expansion over traversable voxels is more efficient. Due to the limited camera FOV, traversable voxels on vertical transition structures may be locally disconnected. Foothold-scale neighborhoods are employed to bridge such small gaps and preserve connectivity across stairs and floors.

For two connected traversable voxels v_i and v_j , the transition cost is defined as

$$J_{\text{step}}(v_i, v_j) = r \|\Delta \mathbf{k}_{ij}\|_2 + \lambda_{\text{esdf}} \Phi(v_j), \quad (15)$$

where r is the voxel resolution, $\Delta \mathbf{k}_{ij}$ is the voxel-index offset from v_i to v_j , and λ_{esdf} is the ESDF safety weight. The safety penalty is

$$\Phi(v_j) = \max \left(0, \frac{d_{\text{safe}} - d_{\text{obs}}(v_j)}{d_{\text{safe}}} \right), \quad (16)$$

where $d_{\text{obs}}(v_j)$ is the ESDF distance to the nearest obstacle and d_{safe} is the safety distance. This cost favors short transitions with sufficient obstacle clearance.

For ATSP construction, geometric costs are obtained by running a multi-target graph search from each source candidate to all other candidates. The returned path lengths directly fill $J_{\text{geo}}(\cdot, \cdot)$ in the ATSP matrix, avoiding repeated pairwise searches. After the next navigation goal is selected, the corresponding 3-D path is recovered by backtracking the parent nodes from the selected target.

C. Execute-Review Mechanism

Execution is treated as a closed-loop validation stage that detects disagreement between the planned motion and the observed robot motion. After a 3-D path is obtained, the system converts path following into executable local actions. In simulation with a discrete action space, the robot selects a local waypoint on the planned path and chooses the next action according to the heading difference between its current yaw and the direction toward this waypoint. In this way, the robot follows the 3-D path through discrete motion primitives rather than directly executing a continuous trajectory.

To monitor execution consistency, the system maintains a predicted pose $\hat{\mathbf{x}}_r^t$ from the action model. After an action is issued, $\hat{\mathbf{x}}_r^t$ is updated according to the corresponding motion primitive. The executed motion is then checked against the odometry pose $\mathbf{x}_{\text{odom}}^t$. If a forward action satisfies

$$\|\mathbf{x}_{\text{odom}}^t - \hat{\mathbf{x}}_r^t\|_2 > \delta_{\text{stuck}}, \quad (17)$$

the robot is considered blocked by an unmodeled obstacle or a local mapping error. When such a failure occurs, the voxel in front of the robot, denoted by v_{front} , is conservatively marked as occupied. The traversable map is then updated, and the planner replans on the new map. If similar failures repeatedly occur near the same target, the corresponding frontier is moved to the dormant set to avoid repeatedly selecting unreachable regions.

Notably, for real-world settings where discrete actions are no longer applicable, this execution layer is replaced by local trajectory optimization to generate safe and continuous spatiotemporal commands.

D. Local Trajectory Optimization

For real-world deployment, the discrete traversable path must be converted into a smooth and dynamically feasible command sequence while preserving the 3-D support-surface profile. The searched 3-D path is refined by a local B-spline trajectory optimizer. The graph-search path is collision-free but piecewise linear due to voxel discretization. The optimizer smooths a local path segment while preserving the ground-following structure encoded by the traversability-aware path.

Let the searched traversable path be $\Gamma = \{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_M\}$, where $\mathbf{x}_i = [x_i, y_i, z_i]^\top \in \mathbb{R}^3$. At each replanning step, a local target $\mathbf{x}_g = [x_g, y_g, z_g]^\top$ is selected from Γ within a look-ahead distance from the current robot state $\mathbf{x}_s = [x_s, y_s, z_s]^\top$.

A local initial segment is generated between $\mathbf{x}_s$ and $\mathbf{x}_g$. The horizontal components are generated by polynomial interpolation according to the current motion state and the selected target, while the height is interpolated along the accumulated horizontal arc length:

$$z_i = z_s + \frac{\sigma_i}{\sigma_K} (z_g - z_s), \quad (18)$$

where σ_i is the accumulated horizontal arc length from $\mathbf{x}_s$ to the i -th sample, and σ_K is the total horizontal arc length of the local segment. The initialized samples are converted into cubic B-spline control points $\{\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_N\}$. Given the knot span Δt , the corresponding velocity and acceleration control points are $\mathbf{v}_i = (\mathbf{Q}_{i+1} - \mathbf{Q}_i)/\Delta t$ and $\mathbf{a}_i = (\mathbf{v}_{i+1} - \mathbf{v}_i)/\Delta t$, respectively. Following [41], the trajectory is optimized by minimizing

$$J_{\text{total}} = \lambda_s J_s + \lambda_o J_o + \lambda_f J_f, \quad (19)$$

where

$$J_s = \sum_{i=1}^{N-2} \left\| \frac{\mathbf{a}_{i+1} - \mathbf{a}_i}{\Delta t} \right\|^2, \quad (20)$$

$$J_o = \sum_{i=0}^N F_{\text{obs}}(d(\mathbf{Q}_i)), \quad (21)$$

and

$$J_f = \sum_{i=0}^{N-1} F_{\text{lim}}(\|\mathbf{v}_i\|, v_{\text{max}}) + \sum_{i=1}^{N-1} F_{\text{lim}}(\|\mathbf{a}_i\|, a_{\text{max}}). \quad (22)$$

Here, J_s penalizes jerk-like variation of acceleration control points, J_o penalizes proximity to obstacles, and J_f enforces dynamic feasibility. The obstacle and limit penalties are defined as $F_{\text{obs}}(d) = \max(d_{\text{min}}^2 - d^2, 0)$ and $F_{\text{lim}}(x, x_{\text{max}}) = \max(x^2 - x_{\text{max}}^2, 0)$, respectively.

Unlike unconstrained 3-D trajectory optimization, the trajectory is not freely deformed away from the traversable surface. Since Γ already provides a ground-following reference over floors and stairs, only the horizontal components of the B-spline control points are optimized. For each control point $\mathbf{Q}_i$, the objective gradient is projected onto the horizontal subspace:

$$\tilde{\nabla}_{\mathbf{Q}_i} J_{\text{total}} = \mathbf{P}_{xy} \nabla_{\mathbf{Q}_i} J_{\text{total}}, \quad \mathbf{P}_{xy} = \text{diag}(1, 1, 0). \quad (23)$$

This projected update improves horizontal smoothness and clearance while preserving the height profile induced by the 3-D traversability-aware path.

VII. RESULTS

A. Experimental Setup

We evaluate TravExplorer in both simulation benchmarks and real-world environments. In simulation, we conduct 4195 episodes across two benchmarks. In the real world, we conduct 50 episodes, including 32 episodes for single-floor object navigation and 18 episodes for cross-floor object navigation. Notably, extensive benchmark comparisons and real-world experiments are demonstrated in the **supplementary videos**.

Simulation benchmark setup. In simulation, we follow the Habitat Challenge 2023 setup [42]. The robot is equipped with a 640×480 RGB-D camera, mounted at a height of 0.88 m. The agent moves forward by 0.25 m per step and rotates by 30° per action. The sensor range is $[0.0 \text{ m}, 5.0 \text{ m}]$, and the success distance is 0.2 m. We conduct experiments on two benchmarks: HM3D [43], with 2000 episodes across 20 high-fidelity scenes and 6 target categories; MP3D [44], with 2195 episodes across 11 scenes and 21 target categories.

Evaluation metrics. Following [42], we use two primary metrics: Success Rate (SR), defined as the percentage of

TABLE I: Comparisons with state-of-the-art methods. The table contrasts learning-based and zero-shot methods on HM3D and MP3D datasets.

Method	Zero-shot	Cross-Floor	HM3D		MP3D	
			SR $\uparrow$	SPL $\uparrow$	SR $\uparrow$	SPL $\uparrow$
SemExp [33]	×	×	–	–	36.0	14.4
PONI [34]	×	×	–	–	31.8	12.1
ZSON [35]	×	×	25.5	12.6	15.3	4.8
CoW [1]	✓	×	–	–	7.4	3.7
ESC [45]	✓	×	39.2	22.3	28.7	14.2
L3MVN [2]	✓	×	50.4	23.1	34.9	14.5
OpenFMNav [46]	✓	×	54.9	24.4	37.2	15.7
VLFM [3]	✓	×	52.5	30.4	36.4	17.5
TriHelper [47]	✓	×	56.5	25.3	–	–
SG-Nav [4]	✓	×	54.0	24.9	40.2	16.0
ApexNav [5]	✓	×	59.6	33.0	39.2	17.8
MFNP [9]	✓	✓	58.3	26.7	41.1	15.4
ASCENT [10]	✓	✓	65.4	33.5	44.5	15.5
TravExplorer	✓	✓	70.0	37.2	48.8	21.1

episodes in which the robot reaches the target object within a predefined success distance, and Success weighted by Path Length (SPL), which penalizes successful episodes by the ratio between the shortest path length and the actual path length. For real-world experiments, where ground-truth shortest path lengths are unavailable, we additionally report Success Penalized by Time (SPT),

$$\text{SPT} = \text{SR} \left(1 - \frac{t}{T_{\text{timeout}}} \right), \quad (24)$$

where t denotes the task completion time and T_{timeout} denotes the predefined timeout threshold. We also report the Average Time (AT) over successful episodes.

Implementation details. For semantic segmentation, we use SAM 3 [48]. The semantic score map is generated by BLIP-2 [39]. The ATSP is solved using the LKH solver [49]. All methods are evaluated under the same benchmark protocol, including identical episode splits, sensor configuration, success criterion, and target categories for fair comparison. All simulation experiments are run on a workstation with an NVIDIA GeForce RTX 4090 GPU and an Intel i9-14900K $\times 32$ CPU.

B. Quantitative Results

Baselines. We compare TravExplorer with representative learning-based, zero-shot, and floor-aware ObjectNav baselines. SemExp [33], PONI [34], and ZSON [35] use learned policies or navigation priors trained with task-specific supervision, which limits zero-shot transfer. CoW [1] performs nearest-frontier exploration without explicit semantic guidance. ESC [45], L3MVN [2], TriHelper [47], and OpenFMNav [46] incorporate semantic maps and LLM-based frontier selection. VLFM [3] and InstructNav [50] construct vision-language value maps for target-directed exploration. SG-Nav [4] uses 3-D scene-graph prompting to improve semantic frontier reasoning. ApexNav [5] adds target-centric semantic fusion for adaptive 2-D exploration, whereas MFNP [9] and ASCENT [10] address multi-floor ObjectNav through floor-aware exploration policies.

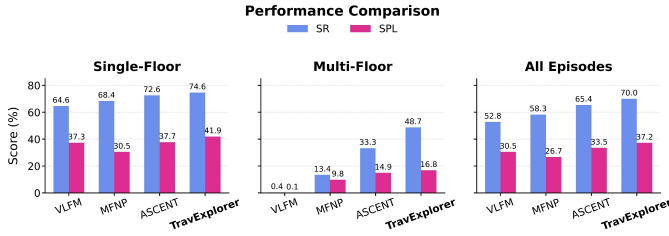


Fig. 7: Performance comparison on HM3D across single-floor, multi-floor, and all-episode settings. We report Success Rate (SR) and Success weighted by Path Length (SPL) for representative zero-shot and floor-aware ObjectNav baselines. TravExplorer achieves the best overall performance with 70.0% SR and 37.2% SPL, while maintaining strong single-floor performance. The improvement is most pronounced in multi-floor episodes, where the unified 3-D traversability representation and cross-floor planning enable more reliable reasoning over stairs, landings, and vertically connected spaces.

Comparisons with SOTAs. Table I reports the comparison with representative learning-based and zero-shot ObjectNav methods on HM3D and MP3D. TravExplorer achieves the best performance on both datasets, with 70.0% SR and 37.2% SPL on HM3D, and 48.8% SR and 21.1% SPL on MP3D. Compared with ASCENT [10], the strongest cross-floor baseline, TravExplorer improves SR/SPL by +4.6%/+3.7% on HM3D and +4.3%/+5.6% on MP3D. Compared with ApexNav [5], a strong zero-shot baseline based on single-floor 2-D exploration, TravExplorer further improves SR/SPL by +10.4%/+4.2% on HM3D and +9.6%/+3.3% on MP3D.

The results show that the proposed method improves both task completion and path efficiency. In particular, most zero-shot baselines, such as VLFM and ApexNav, construct exploration policies on top of 2-D frontier maps, which are effective for planar navigation but cannot explicitly represent vertical connectivity. MFNP and ASCENT introduce floor-aware mechanisms to handle multi-floor environments, yet their representations remain essentially layer-wise and depend on discrete floor transitions. In contrast, TravExplorer represents the environment as a unified 3-D traversable space, where floors, stairs, and intermediate landings are connected through an explicit traversability structure. This representation avoids the ambiguity caused by projecting multi-floor geometry onto 2-D maps and enables the planner to reason directly about cross-floor traversability. The consistent gains on both HM3D and MP3D indicate that explicit 3-D traversability reasoning is beneficial not only for cross-floor reachability, but also for more efficient target-directed exploration.

Performance Analysis Across Floor Scenarios. Fig. 7 further analyzes performance on HM3D by separating single-floor and multi-floor episodes. In single-floor episodes, TravExplorer achieves 74.6% SR and 41.9% SPL, outperforming ASCENT by +2.0% SR and +4.2% SPL. This suggests that introducing a 3-D traversability representation does not degrade conventional planar ObjectNav performance. Instead, the target-centric semantic fusion and value-guided frontier touring help reduce premature target commitments and redundant exploration, leading to higher path efficiency.

The advantage becomes more pronounced in multi-floor episodes. TravExplorer achieves 48.7% SR and 16.8% SPL, exceeding VLFM, MFNP, and ASCENT by +48.3%, +35.3%,

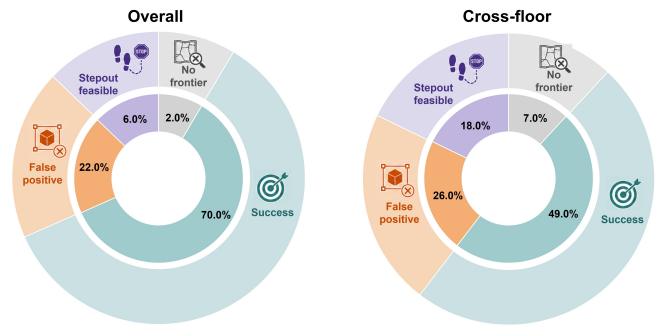


Fig. 8: Failure-mode analysis on HM3D. TravExplorer achieves a 70.0% success rate over all episodes and a 49.0% success rate on cross-floor episodes. The remaining failures are mainly caused by false-positive semantic detections, step-out feasibility violations, and missing frontiers, with cross-floor scenes introducing more feasibility-related challenges due to stairs, landings, and incomplete 3-D observations.

and +15.4% SR, respectively. The comparison also explains the limitations of existing representations. VLFM does not explicitly model floor transitions, making it unsuitable for cross-floor navigation. MFNP relies on floor switching with map reset, which can break spatial continuity and lose useful exploration history. ASCENT improves cross-floor reasoning but still operates on floor-wise 2-D abstractions, where stairs and landings are not represented as continuous traversable structures. By combining active stair perception with foothold-guided 3-D search, TravExplorer maintains continuous traversability across floors and therefore achieves more reliable cross-floor exploration while retaining competitive SPL.

Failure Case Analysis. Outcomes are categorized as follows:

- **Success:** The robot reaches the target and the task is successfully completed.
- **False Positive:** The robot stops near a wrongly detected target, leading to task failure.
- **Stepout Feasible:** The target remains theoretically reachable, but the robot exceeds the step budget before completing the task.
- **No Frontier:** The robot runs out of valid traversable frontiers to explore before reaching the target.

As shown in Fig. 8, the proposed system achieves a success rate of 70% in the complete evaluation setting. Among the remaining failure cases, *False Positive* is the dominant factor, accounting for 22% of all trials. This is mainly caused by the multi-frame inconsistency of the detector, which leads to unstable semantic observations during exploration. With the rapid progress of vision foundation models, this limitation is expected to be substantially alleviated in the near future. Another major cause is annotation error: the agent may find the target, but the task still fails because the object is unlabeled in the dataset, as also observed in [5, 10]. In contrast, exploration-related failures are less frequent. *Stepout Feasible* accounts for 6%, indicating that the target can still be reached in principle, but the robot fails to complete the task within the given step budget. These cases usually occur in large-scale environments or scenes requiring long detours. *No Frontier* only accounts for 2%, suggesting that the traversable-frontier extraction and update strategy can maintain sufficient exploration candidates

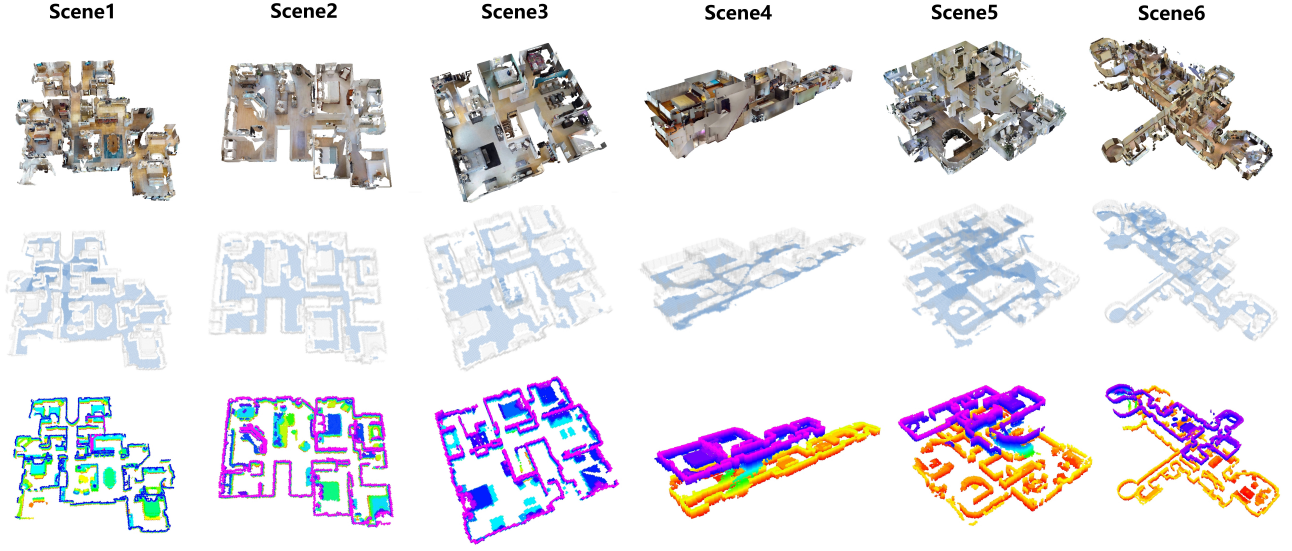


Fig. 9: Qualitative mapping results in six representative indoor scenes. Each column shows the original scene geometry, the reconstructed traversability map, and the corresponding occupancy map from top to bottom. Scenes 1–3 are single-floor scenes, and Scenes 4–6 are multi-floor scenes. The traversability map captures robot-reachable support surfaces, while the occupancy map represents surrounding structures and obstacles, together supporting traversable frontier extraction and 3-D planning.

in most scenes.

The cross-floor setting is significantly more challenging. Compared with the complete setting, *False Positive* remains the largest failure source, increasing to 26%. This indicates that most failures are still caused by annotation errors and the limited capability of the detector, rather than the exploration strategy itself. Importantly, *Stepout Feasible* rises from 6% to 18%. This shows that long-distance targets are difficult to locate using a greedy exploration strategy under a limited step budget. The robot may be attracted to locally promising regions, while the true target requires longer-range reasoning across floors and partially explored areas. Nevertheless, our cross-floor transition strategy is reasonable, since aggressively moving to another floor too early can introduce more revisits and unnecessary detours before sufficient evidence is collected on the current floor. *Frontier Exhausted* also increases from 2% to 7%. This is mainly caused by discontinuous support surfaces, narrow stair transitions, and occluded regions in cross-floor scenes. Moreover, incomplete reconstruction and holes in the habitat simulator lead to inaccurate traversability estimation, which breaks frontier connectivity and causes the robot to exhaust the valid frontiers before finding the correct transition.

C. Qualitative Results

3-D Mapping Performance. We first evaluate the 3-D mapping performance of TravExplorer in diverse indoor layouts. Across 6 representative reconstructions, including both single-floor scenes and multi-floor buildings, the system consistently constructs multiple maps required for 3-D exploration, including traversability and occupancy maps, as shown in Fig. 9. The textured map is only used for scene visualization and is not involved in the planning pipeline. In single-floor scenes, the traversability layer separates walkable regions from walls, furniture, and other non-supporting structures. In multi-floor

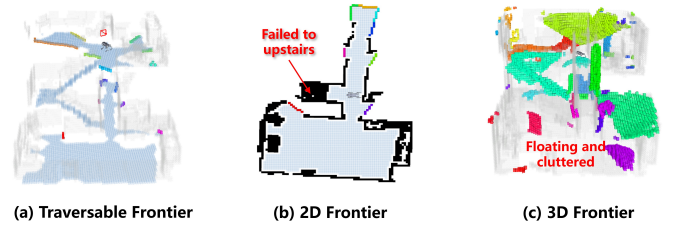


Fig. 10: Qualitative comparison of frontier representations. (a) TravExplorer extracts frontiers on connected traversable support surfaces, yielding reachable exploration goals. (b) 2-D frontiers collapse vertical structures and miss the upstairs transition. (c) Unconstrained 3-D frontiers produce floating and cluttered candidates that are not executable for a ground robot.

scenes, stairs, landings, and vertically overlapping rooms are preserved in a unified 3-D metric frame rather than being collapsed into a top-down grid. This representation provides executable map memory for subsequent exploration, because frontiers and semantic values are attached only to visible and robot-connected traversable regions.

Ablation study of Traversable Frontier. Fig. 10 compares the proposed traversable-frontier representation with conventional 2-D and unconstrained 3-D frontiers. The 2-D frontier representation projects the environment to a planar grid and therefore loses vertical connectivity; this limitation is consistent with the cross-floor result of 2-D-frontier baselines such as VLFM, which achieves only 0.4% SR. In contrast, unconstrained 3-D frontiers preserve height information but generate many floating or obstacle-adjacent candidates that cannot be used as executable viewpoints for a ground robot. TravExplorer extracts frontier clusters directly from connected traversable support surfaces and filters candidates according to obstacle clearance and reachability. The design enables the frontier-based exploration to remain valid in both ordinary planar exploration and cross-floor search. The robot can continuously explore along

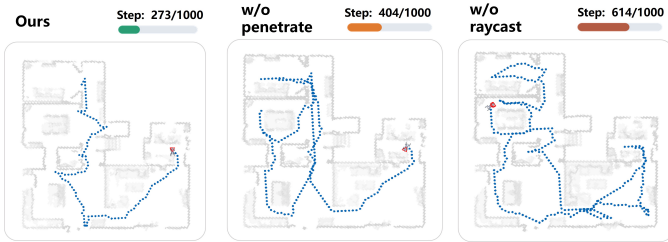


Fig. 11: Mapping ablation on exploration efficiency. TravExplorer reaches the target in 273/1000 steps, while removing penetration-based traversability reasoning or raycast-based visibility checking increases the required exploration to 404/1000 and 614/1000 steps, respectively. The comparison shows that reliable traversability reasoning and visibility-aware frontier updating reduce redundant search and backtracking during target-driven exploration.

reachable support surfaces, identify stair-related transitions, and maintain generalization across different indoor layouts.

Ablation study of Adaptive Mapping. We examine why object-goal exploration cannot rely on semantic segmentation alone. We ablate two key designs in the hybrid and adaptive mapping module: penetration-based raycasting and raycast-based traversability mapping, as visualized in Fig. 11. In a representative planar episode, the complete system reaches the target in 273/1000 steps. Replacing penetration-based raycasting with non-penetrating raycasting increases the required exploration to 404/1000 steps, indicating that penetration improves exploration efficiency for navigation. Since the ObjectNav task does not require complete, accurate scene reconstruction, the robot should avoid carefully scanning every room corner and instead focus on regions that are more relevant to the target. Removing the 2-D raycast-based mapping and relying only on semantic segmentation further increases the required exploration to 614/1000 steps. This degradation is mainly caused by missed and inconsistent semantic predictions, which induce repeated local revisits. These results show that semantic segmentation alone is insufficient, while the proposed hybrid mapping reduces redundant search by combining raycast-based geometric coverage with semantic traversability.

D. Real-World Experiments

We further conduct field experiments to validate the feasibility of the proposed exploration framework across four types of challenging real-world buildings.

Real-world robot setup. In real-world experiments, TravExplorer is deployed on the Unitree Go2 quadruped shown in Fig. 12. The dynamic limits of the quadruped robot are set to $v_m = 1.0\text{ m/s}$, $a_m = 0.25\text{ m/s}^2$, and $\omega_m = 1.0\text{ rad/s}$. The built-in LiDAR of Go2 is removed and replaced with a Livox Mid-360 LiDAR to provide higher-quality point clouds. Our adapted FAST-LIO2 [51] provides high-frequency localization. A forward-looking stereo camera, an Intel RealSense D435i [52], is mounted on a 2-DoF pan-tilt head for active RGB-D perception. The system runs under a ROS-based master-slave architecture. A workstation with an NVIDIA RTX 4090 GPU and an Intel i9-14900HX CPU serves as the ROS master, while the Go2 equipped with an NVIDIA Jetson Orin Nano acts as the slave platform. A portable Wi-Fi router is used for ROS communication between the laptop and the robot.

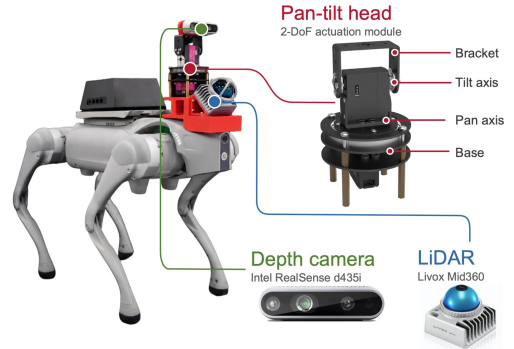


Fig. 12: Hardware platform for real-world deployment. TravExplorer is deployed on a Unitree Go2 quadruped robot equipped with an Intel RealSense D435i depth camera, a Livox Mid-360 LiDAR, and a custom 2-DoF pan-tilt perception head. The LiDAR provides robust localization and geometric sensing, while the RGB-D camera supplies close-range visual and depth observations for occupancy, traversability, and semantic mapping. The pan-tilt head actively adjusts the camera viewpoint to inspect floors, stairs, and occluded regions, supporting FOV-aware active perception and executable cross-floor exploration.

Real-world results. We further conducted real-world experiments to validate the deployability of TravExplorer in challenging indoor buildings. The experiments were performed in four representative environments, as shown in Fig. 13: a multi-floor villa, a complex flat, a cluttered loft, and a spacious office. These scenes cover different spatial scales, floor structures, passage widths, occlusion levels, and dynamic disturbances, including approximately 300 m^2 over two floors in the villa, 165 m^2 on a single floor in the flat, 188 m^2 in the loft-like space, and 450 m^2 over two floors in the office. All trials were started without pre-scanned maps, manually specified intermediate waypoints, or human guidance during execution.

Across the four environments, we conducted 50 physical trials. TravExplorer successfully completed 32 trials, corresponding to a real-world success rate of 64.0%, as shown in Table II. The start locations and target queries were varied within each environment. The queried targets included beds, trash bins, plants, robot-like objects, and other user-specified indoor objects. A trial was counted as successful only if the robot reached a safe pose near the queried target, without manual teleoperation, emergency stop and within a timeout of 300s.

In the multi-floor villa, continuous stair-ascent episodes demonstrate that the proposed 3-D traversability map can represent stairs, landings, and upper-floor surfaces as connected reachable support structures. In the complex flat, the robot used the semantic value map to downweight low-value frontiers and prioritize high-value semantic frontiers related to the queried target, thereby improving exploration efficiency in a multi-room layout. In the cluttered loft, TravExplorer showed accurate local mapping, obstacle avoidance, and narrow-passage navigation under dense furniture and partial visual occlusions. Even with incomplete observations, the robot could actively perceive and recognize targets. In the spacious office, TravExplorer rapidly replanned local collision-free paths around moving pedestrians while maintaining the current semantic search objective or frontier intention. These results validate the effectiveness and deployability of TravExplorer for both single-floor and cross-floor embodied exploration in real indoor environments.

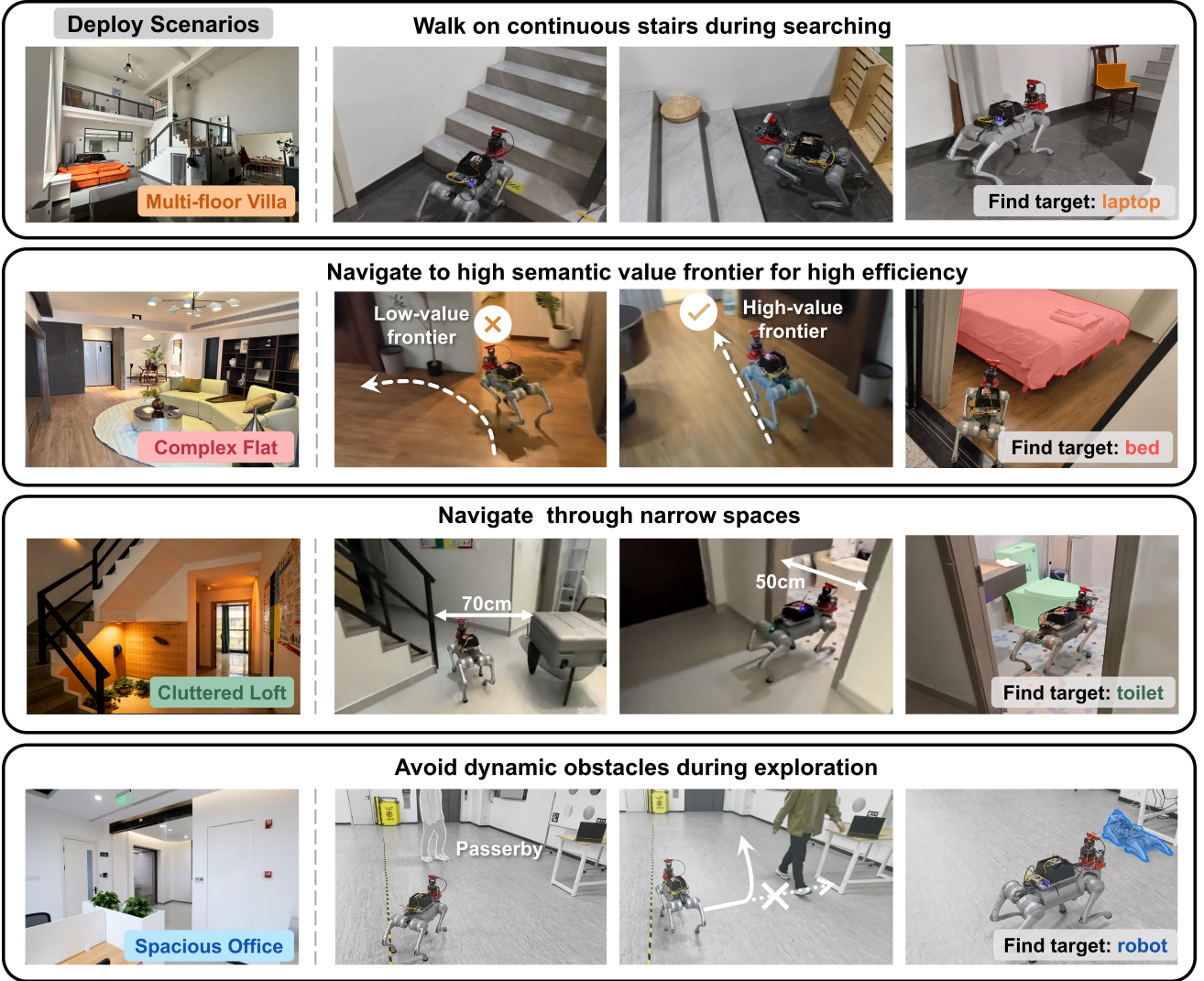


Fig. 13: Real-world deployment results of TravExplorer in four representative indoor environments. The first row shows deployment in a multi-floor villa, where the robot continuously ascends stairs during target search and reaches an executable viewpoint for the queried target, *laptop*. The second row shows a complex flat, where TravExplorer avoids a low-value frontier and selects a high-semantic-value frontier, enabling efficient navigation toward the queried target, *bed*. The third row shows a cluttered loft-like environment, where the robot navigates through narrow passages of approximately 70 cm and 50 cm width while maintaining safe local mapping and obstacle avoidance, and finally searches for the target, *toilet*. The fourth row shows a spacious office area with dynamic pedestrians, where TravExplorer rapidly replans collision-free local paths around moving obstacles while maintaining the semantic search objective for the queried target, *robot*. These deployments demonstrate the real-world capability of TravExplorer in cross-floor traversability reasoning, semantic frontier selection, narrow-space navigation, and dynamic obstacle avoidance.

TABLE II: Quantitative summary of real-world results.

Env.	Trials	Succ.	SR	AT	SPT	Manual	E-stop	Timeout
Villa	7	4	57.1%	203.0 s	18.5%	28.6%	—	14.3%
Flat	5	3	60.0%	46.0 s	50.8%	20.0%	20.0%	—
Loft	18	12	66.7%	57.3 s	53.9%	5.6%	16.7%	11.1%
Office	20	13	65.0%	101.8 s	42.9%	5.0%	15.0%	15.0%
Overall	50	32	64.0%	92.5 s	44.3%	10.0%	14.0%	12.0%

VIII. CONCLUSION

This article presented *TravExplorer*, a traversability-aware embodied exploration framework for zero-shot ObjectNav in cross-floor indoor buildings. The proposed framework addresses the coupling between semantic target search and physically executable navigation by maintaining a unified 3-D traversability map, where floors, stairs, and landings are represented as

connected support surfaces while obstacles are kept separate. Based on this representation, traversable frontiers are incrementally extracted on robot-reachable surfaces, FOV-limited stair observations are completed through active perception, and target-related evidence is accumulated through probabilistic instance memory and spatial value projection. A hierarchical planner further integrates target-aware frontier touring, stair-landmark selection, foothold-guided 3-D graph search, an execute-review mechanism, and vertically constrained local trajectory optimization to convert semantic exploration decisions into executable robot motions.

Extensive simulation and real-world experiments validated the effectiveness of the proposed framework. On HM3D and MP3D, TravExplorer consistently improves both success rate and SPL over representative learning-based, zero-shot, and floor-aware ObjectNav baselines, with the largest gains

observed in multi-floor episodes that require explicit stair and landing connectivity. Real-world deployment on a Unitree Go2 quadruped further demonstrates open-vocabulary target search in single-floor and cross-floor buildings without prior maps or human waypoint guidance, while handling semantic frontier selection, narrow-passage navigation, cross-floor traversal, and local replanning around dynamic obstacles. Future work will focus on extending TravExplorer from a rule-based system toward a general agentic framework, where large vision-language models are used for high-level reasoning, and prediction over unobserved regions.

REFERENCES

- [1] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, “CoWs on PASTURE: Baselines and benchmarks for language-driven zero-shot object navigation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 23 171–23 181.
- [2] B. Yu, H. Kasaei, and M. Cao, “L3MVN: Leveraging large language models for visual target navigation,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 3554–3560.
- [3] N. Yokoyama, S. Ha, D. Batra, J. Wang, and B. Bucher, “VLFM: Vision-language frontier maps for zero-shot semantic navigation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 42–48.
- [4] H. Yin, X. Xu, Z. Wu, J. Zhou, and J. Lu, “SG-Nav: Online 3d scene graph prompting for llm-based zero-shot object navigation,” in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37, 2024.
- [5] M. Zhang, Y. Du, C. Wu, J. Zhou, Z. Qi, J. Ma, and B. Zhou, “ApexNav: An adaptive exploration strategy for zero-shot object navigation with target-centric semantic fusion,” *CoRR*, vol. abs/2504.14478, 2025.
- [6] H. Zhu, Z. Li, Z. Liu, W. Wang, J. Zhang, J. Francis, and J. Oh, “STRIVE: Structured representation integrating VLM reasoning for efficient object navigation,” *CoRR*, vol. abs/2505.06729, 2025.
- [7] H. Zhu, Z. Li, Z. Liu, K. Guo, Z. Lin, Y. Cai, G. Chen, C. Lv, W. Wang, J. Oh, and J. Zhang, “SysNav: Multi-level systematic cooperation enables real-world, cross-embodiment object navigation,” *CoRR*, vol. abs/2603.06914, 2026.
- [8] W. Gai, Y. Gao, Y. Zhou, Y. Xie, Z. Liu, Y. Wu, X. Zhou, F. Gao, and Z. Meng, “USS-Nav: Unified spatio-semantic scene graph for lightweight UAV zero-shot object navigation,” *CoRR*, vol. abs/2602.00708, 2026.
- [9] L. Zhang, H. Wang, E. Xiao, X. Zhang, Q. Zhang, Z. Jiang, and R. Xu, “Multi-floor zero-shot object navigation policy,” *CoRR*, vol. abs/2409.10906, 2024.
- [10] Z. Gong, R. Li, T. Hu, R. Qiu, L. Kong, L. Zhang, G. Zhao, Y. Ding, and J. Liang, “Stairway to success: An online floor-aware zero-shot object-goal navigation framework via LLM-driven coarse-to-fine exploration,” *IEEE Robotics and Automation Letters*, 2026.
- [11] B. Yang, J. Cheng, B. Xue, J. Jiao, and M. Liu, “Efficient global navigational planning in 3-D structures based on point cloud tomography,” *IEEE/ASME Transactions on Mechatronics*, vol. 30, no. 1, pp. 321–332, 2025.
- [12] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proceedings of the 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)*, 1997, pp. 146–151.
- [13] J. Orsulic, D. Miklic, and Z. Kovacic, “Efficient dense frontier detection for 2-D graph SLAM based on occupancy grid submaps,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3569–3576, 2019.
- [14] J. Huang, B. Zhou, Z. Fan, Y. Zhu, Y. Jie, L. Li, and H. Cheng, “FAEL: Fast autonomous exploration for large-scale environments with a mobile robot,” *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1667–1674, 2023.
- [15] Z. Wen, X. Liu, G. Lu, and J. Liu, “Rapid autonomous exploration of large-scale environments for ground robots based on region partitioning,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 13 067–13 073.
- [16] B. Zhou, Y. Zhang, X. Chen, and S. Shen, “FUEL: Fast UAV exploration using incremental frontier structure and hierarchical planning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 779–786, 2021.
- [17] Y. Zhang, X. Chen, C. M. Feng, B. Zhou, and S. Shen, “FALCON: Fast autonomous aerial exploration using coverage path guidance,” *IEEE Transactions on Robotics*, vol. 41, pp. 1365–1385, 2025.
- [18] Y. Tao, X. Liu, I. Spasojevic, S. Agarwal, and V. Kumar, “3D active metric-semantic SLAM,” *IEEE Robotics and Automation Letters*, vol. 9, no. 3, pp. 2989–2996, 2024.
- [19] S. Geng, Z. Ning, F. Zhang, and B. Zhou, “EPIC: A lightweight LiDAR-based UAV exploration framework for large-scale scenarios,” *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 5090–5097, 2025.
- [20] X. Liu, M. Lin, S. Li, G. Xu, Z. Wang, H. Wu, and Y. Liu, “FLARE: Fast large-scale autonomous exploration guided by unknown regions,” *IEEE Robotics and Automation Letters*, vol. 10, no. 11, pp. 12 197–12 204, 2025.
- [21] C. Chen, J. Frey, P. Arm, and M. Hutter, “SMUG planner: A safe multi-goal planner for mobile robots in challenging environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7170–7177, 2023.
- [22] Q. Bi, X. Zhang, S. Zhang, R. Wang, L. Li, and J. Yuan, “LRAE: Large-region-aware safe and fast autonomous exploration of ground robots for uneven terrains,” *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 186–11 193, 2024.
- [23] Y. Jia, W. Tang, H. Sun, J. Yang, B. Liu, and C. Wang, “Portable planner for enhancing ground robots exploration performance in unstructured environments,” *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 9295–9302, 2024.
- [24] L. Zheng, M. Wei, R. Mei, K. Xu, J. Huang, and H. Cheng, “AAGE: Air-assisted ground robotic autonomous exploration in large-scale unknown environments,” *IEEE Transactions on Robotics*, vol. 41, pp. 1918–1937, 2025.
- [25] P. Fankhauser, M. Bloesch, and M. Hutter, “Probabilistic terrain mapping for mobile robots with uncertain localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 3019–3026, 2018.
- [26] T. Miki, L. Wellhausen, R. Grandia, F. Jenelten, T. Homberger, and M. Hutter, “Elevation mapping for locomotion and navigation using GPU,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 2273–2280.
- [27] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, “Perceptive locomotion through nonlinear model-predictive control,” *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3402–3421, 2023.
- [28] L. Xu, K. Chai, Z. Han, H. Liu, C. Xu, Y. Cao, and F. Gao, “An efficient trajectory planner for car-like robots on uneven terrain,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2853–2860.
- [29] J. Wang, L. Xu, H. Fu, Z. Meng, C. Xu, Y. Cao, X. Lyu, and F. Gao, “Towards efficient trajectory generation for ground robots beyond 2D environment,” *CoRR*, vol. abs/2302.03323, 2023.
- [30] M. Zhang, Z. Tian, Y. Xia, C. Xu, F. Gao, and Y. Cao, “Efficient trajectory generation based on traversable planes in 3D complex architectural spaces,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 14 513–14 519.
- [31] Y. Li, K. Chen, Y. Wang, W. Zhang, J. Wang, H. Chen, and Y. Liu, “Real-time multilevel terrain-aware path planning for ground mobile robots in large-scale rough terrains,” *IEEE Transactions on Robotics*, vol. 41, pp. 4159–4179, 2025.
- [32] Y. Wang, W. Zhang, Y. Li, J. Wang, and H. Chen, “TiFA: A terrain-informed navigation framework for articulated tracked robots in rescue missions,” *The International Journal of Robotics Research*, 2025, onlineFirst.
- [33] D. S. Chaplot, D. P. Gandhi, A. Gupta, and R. Salakhutdinov, “Object goal navigation using goal-oriented semantic exploration,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 4247–4258.
- [34] S. K. Ramakrishnan, D. S. Chaplot, Z. Al-Halah, J. Malik, and K. Grauman, “PONI: Potential functions for objectgoal navigation with interaction-free learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 18 868–18 878.
- [35] A. Majumdar, G. Aggarwal, B. Devnani, J. Hoffman, and D. Batra, “ZSON: Zero-shot object-goal navigation using multimodal goal embeddings,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [36] T. Chabal, S. Chen, J. Ponce, and C. Schmid, “FOM-Nav: Frontier-object maps for object goal navigation,” *CoRR*, vol. abs/2512.01009, 2025.
- [37] E. Padilla-Cerdio, B. Sun, M. Pollefeys, and H. Blum, “OpenFrontier: General navigation with visual-language grounded frontiers,” *CoRR*, vol. abs/2603.05377, 2026.
- [38] D. Batra, A. Gokaslan, A. Kembhavi, O. Maksymets, R. Mottaghi, M. Savva, A. Toshev, and E. Wijmans, “ObjectNav revisited: On evaluation

- of embodied agents navigating to objects,” *CoRR*, vol. abs/2006.13171, 2020.
- [39] J. Li, D. Li, S. Savarese, and S. C. H. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19 730–19 742.
 - [40] Z. Meng, H. Qin, Z. Chen, X. Chen, H. Sun, F. Lin, and M. H. A. Jr., “A two-stage optimized next-view planning framework for 3-D unknown environment exploration, and structural reconstruction,” *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1680–1687, 2017.
 - [41] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, “Ego-planner: An esdf-free gradient-based local planner for quadrotors,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, 2021.
 - [42] Habitat Team, “Habitat navigation challenge 2023,” <https://github.com/facebookresearch/habitat-challenge>, 2023, accessed: 2026-05-17.
 - [43] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. M. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, “Habitat-Matterport 3D Dataset (HM3D): 1000 large-scale 3d environments for embodied AI,” in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
 - [44] A. X. Chang, A. Dai, T. Funkhouser, M. Halber, M. Nießner, M. Savva, S. Song, A. Zeng, and Y. Zhang, “Matterport3D: Learning from RGB-D data in indoor environments,” in *2017 International Conference on 3D Vision (3DV)*, 2017, pp. 667–676.
 - [45] K. Zhou, K. Zheng, C. Pryor, Y. Shen, H. Jin, L. Getoor, and X. E. Wang, “ESC: Exploration with soft commonsense constraints for zero-shot object navigation,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 42 829–42 842.
 - [46] Y. Kuang, H. Lin, and M. Jiang, “OpenFMNav: Towards open-set zero-shot object navigation via vision-language foundation models,” in *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, 2024, pp. 338–351.
 - [47] L. Zhang, Q. Zhang, H. Wang, E. Xiao, Z. Jiang, H. Chen, and R. Xu, “TriHelper: Zero-shot object navigation with dynamic assistance,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
 - [48] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. S. Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, “SAM 3: Segment anything with concepts,” *CoRR*, vol. abs/2511.16719, 2025.
 - [49] K. Helsgaun, “An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems,” Roskilde University, Roskilde, Denmark, Technical Report, 2017. [Online]. Available: http://www.akira.ruc.dk/~keld/research/LKH-3/LKH-3_REPORT.pdf
 - [50] Y. Long, W. Cai, H. Wang, G. Zhan, and H. Dong, “InstructNav: Zero-shot system for generic instruction navigation in unexplored environment,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2049–2060.
 - [51] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “Fast-lid2: Fast direct lidar-inertial odometry,” *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
 - [52] Intel Corporation, “Intel RealSense Depth Camera D435i,” <https://www.intel.com/content/www/us/en/products/sku/190004/intel-realsense-depth-camera-d435i.html>, 2026, accessed: 2026-05-17.